

Partially Blind BBS Signatures for Unlinkable Decentralized Identity

A Succinct On-Chain Verifiable Credential Framework

Abstract

Decentralized Identity (DID) systems aim to provide users with self-sovereign control over their credentials. However, verifying identities on public blockchains often risks leaking sensitive attributes and transaction histories, whereas relying on heavy generic cryptographic protocols leads to prohibitive on-chain costs. In this paper, we propose **BBS-DID**, a verifiable credential framework that integrates partially blind BBS signatures with Non-Interactive Zero-Knowledge (NIZK) proofs. By shifting complex identity validation off-chain while blinding the core identity-binding elements, the scheme ensures on-chain unlinkability without losing attribute auditability. To prevent Sybil attacks and credential transfer, we integrate a deterministic nullifier tied to Soulbound Tokens (SBTs). We formalize the scheme’s system model and establish its security regarding existential unforgeability, signer-blindness, and zero-knowledge. Performance results show that this construction maintains minimal on-chain gas consumption, offering a practical deployment path for privacy-preserving verifiable credentials in Web3 environments.

Keywords: Decentralized Identity, Verifiable Credentials, BBS Signatures, Zero-Knowledge Proofs, Blockchain, Blind Signature

1 Introduction

1.1 Motivation

The transition to Web3 applications has driven digital identity management toward user-centric models. Decentralized Identity (DID) [RS⁺22] systems and Verifiable Credentials (VC) [SLL⁺22] enable individuals to manage their digital footprints independently, reducing reliance on centralized identity providers and limiting exposure to data breaches.

Anonymous Credentials (AC) [CL04] serve as a core component for privacy-preserving authentication. The BBS signature scheme, originally proposed by Boneh, Boyen, and Shacham [BBS04] and adapted for modern credentials [CDL16, TZ23], is widely used in this domain. Because BBS signatures natively support selective disclosure and zero-knowledge proofs, they are being integrated into W3C standards and frameworks such as the EU Digital Identity Wallet (eIDAS 2.0).

However, deploying these identity mechanisms on public blockchains presents privacy and scalability challenges. While generic zero-knowledge arguments (e.g., zk-SNARKs [Gro16]), zero-knowledge verifiable encryption (e.g., SAVER [LCKO19]), and auditing extensions [ZLB24] can provide on-chain privacy, directly applying them to DID verification usually incurs excessively high gas costs. In addition, standard on-chain credentials risk being traded like generic digital assets, defeating the non-transferability requirement of identity attributes. Soulbound Tokens (SBTs) [WOB22] restrict transferability by binding tokens

to specific blockchain addresses, but enabling privacy-preserving interactions for SBTs remains computationally challenging.

SBTs provide utility for on-chain identity by enabling applications such as KYC-gated DeFi protocols, reputation systems for decentralized development communities, and role-based access control. They bind authenticated attributes to a user’s blockchain address. Adding privacy layers to SBTs allows users to prove these attributes on-chain without explicitly revealing the underlying data.

1.2 Limitations of Prior Work

Existing solutions for unlinkable decentralized identities face significant trade-offs. Recent frameworks like OpenAC [ENR⁺26] utilize general-purpose zk-SNARKs to preserve issuer key compatibility; however, they suffer from prohibitive on-chain verification costs due to the lack of native EVM precompiles for their verification algorithms, resulting in gas overheads often exceeding 1 million. While threshold BBS+ schemes [DKL⁺23] attempt to distribute trust, they introduce substantial communication complexity during issuance and are primarily designed for off-chain verification, making them inefficient for high-frequency on-chain applications. Furthermore, standard BBS signatures [BBS04] are inherently *linkable*, allowing issuers to track users across different sessions—a critical privacy gap that prior art fails to address without sacrificing efficiency.

1.3 Practical Web3 authentication patterns and limitations

Current implementations link off-chain accounts to on-chain addresses through relatively lightweight methods, such as OAuth-based account linking, social media proofs, Sign-In-With-Ethereum (SIWE), and centralized KYC attestations. While functional, these methods suffer from dependencies on centralized providers, risks of account takeover, permanent on-chain linkability, and vulnerability to Sybil attacks. Conversely, full zero-knowledge solutions remain costly.

BBS-DID addresses these practical limitations. By utilizing partially blind BBS issuance (hiding address and nullifier data from the Signer) alongside deterministic address-bound nullifiers, BBS-DID maintains unlinkability between the issuance phase and the on-chain presentation. It enforces a one-time issuance of SBTs and offloads the intensive attribute verification to the off-chain Signer to keep on-chain execution succinct. The design acknowledges certain trade-offs, such as reliance on the Signer for availability (which threshold issuance can mitigate) and the additional client-side computation required for blinding.

Figure 1 illustrates the system architecture and the boundary between off-chain identity validation and on-chain issuance.

1.4 Our Contributions

To resolve these challenges, we propose **BBS-DID**, a framework combining partially blind BBS signatures and address-bound deterministic nullifiers to support anonymous and Sybil-resistant decentralized identity checks on-chain. Our main contributions are:

1. **Decoupled DID Architecture:** We provide a verification model that delegates intensive attribute validation (e.g., KYC processes) to an off-chain issuer. The on-chain smart contracts are then only responsible for succinct cryptographic verification.
2. **On-chain Unlinkability:** We apply a partially blind extension to BBS issuance. By blinding the user’s specific identity parameters from the Signer, our scheme breaks the linkability, ensuring that the on-chain NIZK proof verification cannot be correlated.

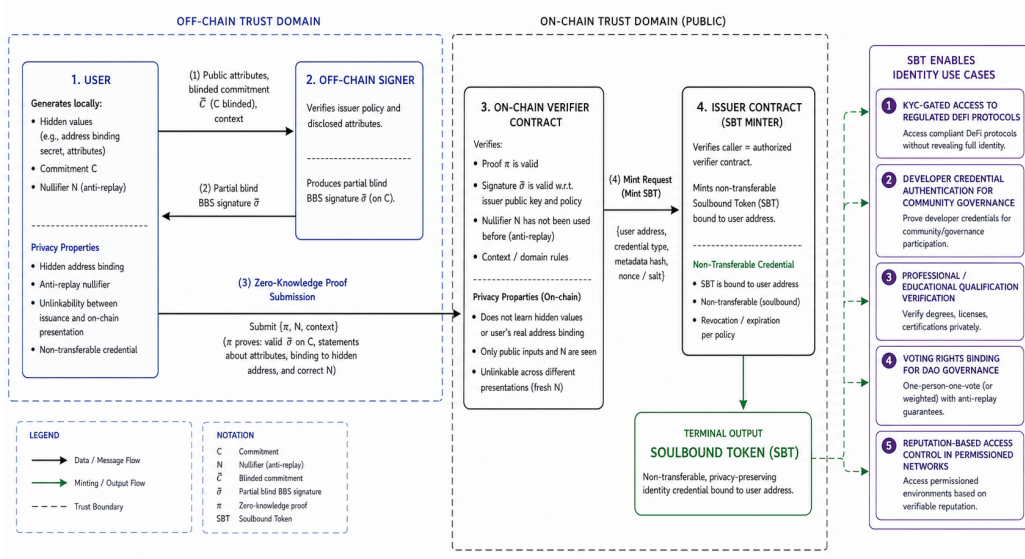


Figure 1: System architecture of our scheme.

3. **Sybil Resistance via SBTs:** Constructing upon nullifier concepts from privacy-preserving cryptocurrencies (e.g., Zcash [BSCG⁺14]) and mixing protocols (e.g., Tornado Cash [PSS19]), we bind zero-knowledge operations to deterministic nullifiers. This approach translates verifiable credentials into privacy-preserving Soulbound Tokens, preventing credential replay and restricting multi-address double-spending.

1.5 Paper Organization

The remainder of this paper is structured as follows. Section 2 reviews the underlying cryptographic primitives, including bilinear pairings and BBS signatures. Section 3 formalizes the system model and security definitions. Section 4 outlines the construction and algorithms of the BBS-DID scheme. Section 5 covers the security proofs. Section 6 details the performance and gas consumption of our smart contract implementation, and Section 7 concludes the paper.

2 Preliminaries

2.1 Bilinear Pairings

Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be two cyclic cryptographic groups of prime order p , and let $\mathbb{G}_T$ be a target cyclic group of the same order p . An asymmetric bilinear pairing is a computable map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ satisfying the following properties:

- **Bilinearity:** For all $U \in \mathbb{G}_1, V \in \mathbb{G}_2$ and $a, b \in \mathbb{Z}_p$, $e(aU, bV) = e(U, V)^{ab}$.
- **Non-degeneracy:** If G_1 generates $\mathbb{G}_1$ and G_2 generates $\mathbb{G}_2$, then $e(G_1, G_2)$ generates $\mathbb{G}_T$.
- **Computability:** There exists an efficient algorithm to compute $e(U, V)$ for any $U \in \mathbb{G}_1, V \in \mathbb{G}_2$.

The security of schemes constructed over these groups typically relies on standard assumptions for pairing-friendly curves, such as the q -Strong Diffie-Hellman (q -SDH) assumption [CDL16, TZ23].

2.2 BBS Signatures

The BBS signature scheme, originally proposed by Boneh, Boyen, and Shacham, is a short signature scheme designed for signing vectors of messages. A standard BBS signature on a message vector $\vec{m} = \{m_1, \dots, m_n\}$ consists of a pair $(A, e) \in \mathbb{G}_1 \times \mathbb{Z}_p$. The scheme operates over the bilinear pairing parameters with three core algorithms:

- **KeyGen:** The signer samples a random secret key $x \xleftarrow{\$} \mathbb{Z}_p$ and computes the public key $X = x \cdot G_2$. Public parameters include independent generators $H_1, \dots, H_n \in \mathbb{G}_1$ for mapping messages.
- **Sign:** To sign a message vector $\vec{m} \in \mathbb{Z}_p^n$, the signer samples a random scalar $e \xleftarrow{\$} \mathbb{Z}_p$, and computes $A = \frac{1}{x+e}(G_1 + \sum_{i=1}^n m_i H_i)$. The signature is $\sigma = (A, e)$.
- **Verify:** Given the signature $\sigma = (A, e)$, public key X , and messages $\vec{m}$, the verifier checks the pairing relation: $e(A, X + eG_2) = e(G_1 + \sum_{i=1}^n m_i H_i, G_2)$.

Recent work by Tessaro and Zhu [TZ23] formalized the security of the core BBS signature, proving its strong unforgeability under the q -SDH assumption in the standard model. This resolved previous theoretical gaps regarding the short form of BBS compared to the BBS+ variant [CDL16], which requires an additional scalar s to form $\sigma = (A, e, s)$. We utilize BBS signatures because they inherently support selective disclosure, allowing a prover to demonstrate possession of a valid signature on $\vec{m}$ while keeping a subset of the messages (e.g., blind factors) hidden [DKL⁺23, TZ23].

2.3 Zero-Knowledge Proofs & Fiat-Shamir Heuristic

Zero-Knowledge Proofs of Knowledge (ZKPoK) enable a prover to convince a verifier that they possess a secret witness satisfying a mathematical statement, without revealing the witness itself. In our scheme, ZKPoK is used to prove possession of a valid BBS signature and selectively disclose attributes while maintaining zero-knowledge properties [TZ23].

Interactive zero-knowledge protocols can be converted into Non-Interactive Zero-Knowledge (NIZK) proofs using the Fiat-Shamir heuristic [FS87]. By deriving the verifier's challenge c via a collision-resistant hash function applied to the public statement (context, revealed messages, system parameters) and the prover's commitments, the protocol becomes non-interactive. This transformation makes the proofs publicly verifiable and suitable for on-chain smart contract execution.

2.4 Soulbound Tokens (SBT)

Soulbound Tokens (SBTs) are non-transferable, identity-bound digital tokens. SBTs link explicit attestations or verifiable credentials directly to specific blockchain addresses. Through standardized interfaces such as ERC-5114 or ERC-5484, smart contracts enforce transfer restrictions, tying the token permanently to an owner. In our architecture, SBTs serve as the on-chain representation of off-chain verified credentials, allowing decentralized applications to verify identity properties directly without repeating the underlying zero-knowledge proof validations.

3 System Model and Security Definitions

3.1 System Entities

Our proposed BBS-DID framework consists of four primary entities, designed to decouple intensive cryptographic computations from on-chain execution:

- **User (Prover)**: A participant requesting a verifiable credential (e.g., a decentralized passport) to authorize on-chain interactions. The User interacts with the Signer off-chain to obtain the credential and generates a Non-Interactive Zero-Knowledge (NIZK) proof to authenticate anonymously on-chain.
- **Signer (Trusted Third Party / TTP)**: An off-chain authorized node holding a private signing key. It is responsible for verifying user attributes and issuing partially blind BBS signatures over the user's commitments.
- **Verifier (Relayer Contract)**: An intermediary smart contract that performs on-chain authentication of the NIZK proofs and triggers callback functions.
- **Issuer (DApp Contract)**: An on-chain target contract that manages the minting of address-bound SBTs upon successful verification.

3.2 Algorithms and Syntax

The BBS-DID protocol is formalized as a tuple of five polynomial-time algorithms $\Pi = (\text{Init}, \text{Apply}, \text{Sign}, \text{Prove}, \text{Verify})$:

- $\text{Init}(1^\lambda) \rightarrow (\text{pp}, \text{sk}, \text{pk})$: Executed by the Issuer and Signer. Takes the security parameter 1^λ and outputs public parameters pp (including pairing groups and standard generators), the Signer's secret key sk , and the corresponding public key pk .
- $\text{Apply}(\text{pp}, \text{addr}, \vec{m}) \rightarrow (\mathcal{C}_2, \text{st}_{\text{user}})$: Executed by the User. Takes the user's blockchain address addr and attribute vector $\vec{m}$. It generates blinding factors, computes a blinded commitment $\mathcal{C}_2$ and nullifier $\mathcal{N}$, and records an internal state st_{user} .
- $\text{Sign}(\text{sk}, \mathcal{C}_2, \vec{m}) \rightarrow (\sigma', \perp)$: Executed by the Signer. Verifies the public attributes in $\vec{m}$. If approved, it signs the commitment $\mathcal{C}_2$ using sk to produce a partial signature σ' . Otherwise, it outputs $\perp$.
- $\text{Prove}(\text{pp}, \text{pk}, \sigma', \text{st}_{\text{user}}, M) \rightarrow \pi$: Executed by the User. Unblinds σ' to recover the canonical signature, and uses the internal state to generate an NIZK proof bound to an optional context message M . Outputs the proof π .
- $\text{Verify}(\text{pp}, \text{pk}, \pi, M, \mathcal{N}) \rightarrow \{0, 1\}$: Executed by the on-chain Verifier. Validates the proof π with respect to the nullifier $\mathcal{N}$ and context M . Returns 1 (accept) to trigger SBT minting, or 0 (reject).

3.3 Security Requirements

A secure Decentralized Identity scheme operating over public blockchains must satisfy four primary security properties:

3.3.1 Existential Unforgeability against Chosen Message Attacks (EUF-CMA)

No Probabilistic Polynomial-Time (PPT) adversary $\mathcal{A}$, even with access to a signing oracle $\mathcal{O}_{\text{Sign}}$, should be able to produce a valid NIZK proof π^* for an attribute vector $\vec{m}^*$ that was not previously authorized by the Sign procedure. This prevents the fabrication of valid credentials.

3.3.2 Signer-Blindness and On-chain Unlinkability

The verification process must not leak identifying correlations. Even a malicious Signer possessing all historical blinded queries $\mathcal{C}_2$ must be unable to link a valid on-chain proof π to its corresponding issuance transcript with a probability non-negligibly better than random guessing.

3.3.3 Sybil Resistance and Non-Transferability (Double-Spending Prevention)

Leveraging the deterministically bound nullifier $\mathcal{N}$, the scheme must prevent credential replay or token double-spending. Using a single authorized credential st_{user} to authenticate multiple distinct blockchain addresses must be computationally equivalent to solving the Discrete Logarithm (DL) problem, thus enforcing a strict 1-to-1 token minting constraint.

3.3.4 Zero-Knowledge Capability

The public NIZK proof π published on-chain must reveal no information about the underlying attributes beyond the fact that they satisfy the required conditions. Observers learn nothing other than the binary validity evaluation of the Verify function.

4 Proposed Scheme

The core of our protocol leverages the algebraic properties of the BBS Signature framework [TZ23] to enable selective disclosure and privacy-preserving identity claims.

Figure 2 gives an overview of the end-to-end protocol flow, including off-chain signing, proof generation, and on-chain verification.

4.1 Design Rationale

Our scheme’s architecture is developed to balance security, user privacy, and on-chain efficiency. The design rationale centers on the following objectives:

1. **Decoupled Verification to Minimize On-chain Footprint:** To avoid the high gas costs of on-chain data processing, complex identity validations (e.g., KYC) are performed by an off-chain Trusted Third Party (*Signer*). The blockchain smart contracts (*Verifier/Issuer*) are restricted to evaluating succinct zero-knowledge proofs and minting the resulting identity-bound Soulbound Tokens (SBTs).
2. **Cryptographic Unforgeability via BBS Signatures:** Credential authenticity relies on the unforgeability of BBS signatures. Only the authorized Signer, holding the secret key x , can compute a valid signature over the attributes, preventing credential fabrication.
3. **Sybil Resistance and Anti-Replay via Bound Nullifiers:** To prevent sharing and replay attacks, each credential is bound to the user’s destination blockchain address (m_{Addr}). This is enforced through a deterministic nullifier: $\mathcal{N} = \text{Hash}(0x\text{UserAddr} \parallel$

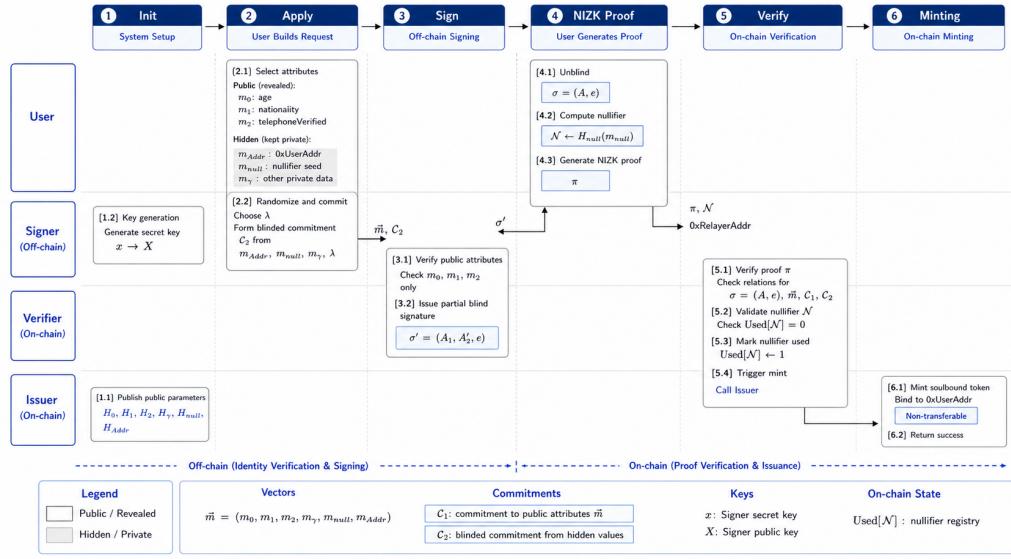


Figure 2: Protocol flow of our scheme from initialization and partially blind signing to NIZK verification and SBT minting.

m_{null}). By recording N' on-chain, the protocol establishes a strict one-to-one mapping between a signature and a minted credential, requiring users to request a new signature for subsequent issuances.

- Mitigating Signer Traceability via Commitments:** If the user transmits the nullifier seed (m_{null}) and destination address (m_{Addr}) in plaintext, the Signer learns the user’s on-chain identity. To prevent this, the protocol hides these identity-binding components within a cryptographic commitment using an ephemeral scalar m_γ as randomness.
- Addressing Linkability in Standard Commitments:** Committing to the variables is insufficient for unlinkability if the Signer issues a standard signature directly over the commitment tuple. The algebraic structure would remain deterministic, allowing the Signer to correlate the on-chain zero-knowledge proof (π) with the initial issuance transcript.
- Achieving Anonymity with Partially Blind Signatures:** To break this linkage, we use a *partial blind signature* mechanism. The user applies a blinding factor (λ) to mask the commitment during the signing request. The identity-binding and anti-Sybil components ($m_{Addr}, m_{null}, m_\gamma$) are blinded, while the explicit attributes ($m_0, m_1, \dots$) remain in plaintext. This allows the Signer to validate the factual claims prior to signing while providing the user with unlinkability during on-chain execution.

4.2 System Architecture

- User:** The participant seeking a verifiable credential (e.g., a decentralized passport) to authorize on-chain interactions. The user registers a USERID (e.g., email, phone number) with the Signer.
- Issuer:** An on-chain smart contract deployed by the DApp, responsible for minting

an address-bound soulbound token (SBT, complying with ERC-5114 or ERC-5484) to the verified User.

- **Signer:** An off-chain Trusted Third Party (TTP) node that evaluates user identity attributes and issues valid BBS signatures.
- **Verifier:** An intermediary contract that authenticates the user's NIZK proofs and triggers automated callbacks to the Issuer contract to execute minting protocols.

4.3 Symbol Definitions

- $\langle \dots \rangle$: Denotes a relation of the data in the braces.
- $0xUserAddr$ (mapped to scalar m_{Addr}): The user's address on the blockchain.
- $\mathcal{N}$: The nullifier, a unique identifier derived from the user's address and a random seed, used to prevent double-spending of credentials.
- $\vec{m}$: A vector of public message components representing explicit identity constraints.
- **TokenID:** The final identifier of the minted SBT, deterministically derived from the nullifier $\mathcal{N}$ to ensure a strict one-to-one mapping between credentials and tokens.

4.4 Message Coding Definitions

To mathematically formalize the constraints, we designate specific properties to unique discrete message generators:

Variable	Type	Message Contents / Identity Constraints
m_0, H_0	Public	Explicit Constraint 1 (e.g., <code>Hash("Is_Adult=true")</code>)
m_1, H_1	Public	Explicit Constraint 2 (e.g., <code>Hash("Nationality=US")</code>)
m_2, H_2	Public	Explicit Constraint 3 (e.g., <code>Hash("TelephoneVerified=true")</code>)
$\dots$	$\dots$	$\dots$
$m_{\ell-1}, H_{\ell-1}$	Public	Explicit Constraint ℓ
m_γ, H_γ	Hidden	Ephemeral Blinding Factor (Concealed from the Signer)
$m_{\text{null}}, H_{\text{null}}$	Hidden	Nullifier seed (Concealed from the Signer)
$m_{\text{Addr}}, H_{\text{Addr}}$	Hidden	User's Address (Concealed from the Signer)

4.5 Protocol Workflow

4.5.1 Phase 1: Initialization (Init)

- **Issuer** generates $\ell + 3$ independent random generators:

$$H_0, H_1, \dots, H_{\ell-1}, H_\gamma, H_{\text{null}}, H_{\text{Addr}} \xleftarrow{\$} \mathbb{G}_1$$

and a mapping between the explicit identity constraints and the corresponding message generators, then publishes them as public parameters.

- **Signer** executes the KeyGen algorithm to sample a secret key $x \xleftarrow{\$} \mathbb{Z}_p$ and publishes the corresponding public key $X = x \cdot G_2$.
- **Signer** or designated **Issuer** deploys the *Verifier* contract, embedding the public parameters $\{H_i\}, H_\gamma, H_{\text{null}}, H_{\text{Addr}}$, and public key X .

Current Status

- **Public Parameters:** $H_0, \dots, H_{\ell-1}, H_\gamma, H_{\text{null}}, H_{\text{Addr}}, X, \vec{m}$
- **Signer Status:** Secret key x held locally.

4.5.2 Phase 2: User Application (Apply)

- **User:**
 1. Retrieves mandatory parameters and identity constraints ($\vec{m}$) using **view** calls against the target contract.
 2. Samples random scalars $m_{\text{null}}, m_\gamma, \lambda \xleftarrow{\$} \mathbb{Z}_p$, and derives the scalar parameter m_{Addr} from $0x\text{UserAddr}$.
 3. Computes the unique deterministic nullifier:
 $\mathcal{N} = \text{Hash}(0x\text{UserAddr} \parallel m_{\text{null}})$.
 4. Computes a foundational commitment covering the identity variables:
 $\mathcal{N}' = m_{\text{Addr}} \cdot H_{\text{Addr}} + m_{\text{null}} \cdot H_{\text{null}} + m_\gamma \cdot H_\gamma$
 5. Applies the blinding factor λ to obscure the commitment from the Signer without revealing m_{Addr} or m_{null} :
 $\mathcal{C}_2 = \lambda \cdot \mathcal{N}' = \lambda \cdot (m_{\text{Addr}} \cdot H_{\text{Addr}} + m_{\text{null}} \cdot H_{\text{null}} + m_\gamma \cdot H_\gamma)$
 6. Aggregates the plaintext evidence vector $\vec{m} = \{m_0, \dots, m_{\ell-1}\}$ and transmits $(\vec{m}, \mathcal{C}_2)$ **off-chain** to the **Signer**.

Current Status

- **Public Parameters:** $H_{\text{null}}, X, \vec{m}$
 - **Verifier Contract Context:** $\langle \mathcal{N} \rangle$
- **Signer Scope:** $x, \langle \text{UserID}, \vec{m}, \mathcal{C}_2 \rangle$ pending review.
- **User Scope:** $0x\text{UserAddr}, m_{\text{null}}, m_\gamma, \lambda, \mathcal{N}$ generated.

4.5.3 Phase 3: Partial Blind Signature Issuance (Sign)

- **Signer:**
 1. Verifies that the explicit constraints ($\vec{m}$) correspond to the registered attributes of the UserID. If any check fails, the process is aborted.
 2. Computes the public commitment segment for the known attributes:
 $\mathcal{C}_1 = G_1 + \sum_{i=0}^{\ell-1} m_i \cdot H_i \in \mathbb{G}_1$
 3. Samples $e \xleftarrow{\$} \mathbb{Z}_p$ and computes the first partial signature component:
 $A_1 = \frac{1}{x+e} \cdot \mathcal{C}_1$
 4. Applies the same e scalar to the User's blinded commitment to finalize the partial signature:
 $A'_2 = \frac{1}{x+e} \cdot \mathcal{C}_2$
 5. Returns the partial signature $\sigma' = (A_1, A'_2, e)$ to the User to conclude the issuance phase.

Current Status

- **Public Parameters:** $H_{\text{null}}, X, \vec{m}$
- **Signer Scope:** $x, \langle \text{UserID}, \mathcal{C}_2, \sigma' \rangle$
- **User Scope:** $0x\text{UserAddr}, m_{\text{null}}, m_{\gamma}, \lambda, \mathcal{N}, \sigma'$

4.5.4 Phase 4: Proof Generation (NIZK Proof)• **User:**

1. Unblinds the component using the inverse factor λ^{-1} :

$$A_2 = A'_2 \cdot \lambda^{-1} = \frac{m_{\text{Addr}} \cdot H_{\text{Addr}} + m_{\text{null}} \cdot H_{\text{null}} + m_{\gamma} \cdot H_{\gamma}}{x + e}$$
2. Reconstructs the algebraically canonical BBS signature over all messages:
 $\sigma = (A, e) = (A_1 + A_2, e)$
3. Constructs bounds and proofs to formulate a verifiable **NIZK Proof** $\pi = (\bar{A}, \bar{B}, U, s, t, u_{\gamma})$ demonstrating possession of the signature over $(\vec{m}, m_{\text{null}}, m_{\gamma})$ while preserving zero-knowledge for (m_{γ}, A, e) .
 Using index arrays for tracking explicit readable vectors $J = \{0, 1, \dots, \ell - 1, \text{null}, \text{Addr}\}$ versus hidden identifiers $I = \{\gamma\}$:

$$\begin{aligned} \mathcal{C} &= \mathcal{C}_J + \mathcal{C}_I \\ &= G_1 + \sum_{j \in J} m_j \cdot H_j + m_{\gamma} \cdot H_{\gamma} \end{aligned}$$

Executes the Fiat-Shamir transformation sequence:

- $\bar{A} = r \cdot A$ (Randomized signature point representation)
 - $\bar{B} = r \cdot \mathcal{C} - r \cdot e \cdot A$
 - $U = \alpha \cdot \mathcal{C}_J + \beta \cdot \bar{A} + \delta_{\gamma} \cdot H_{\gamma}$
 - $c = H(\text{ctx} \parallel \vec{m}_J \parallel \bar{A} \parallel \bar{B} \parallel U)$
 - $s = \alpha + r \cdot c, \quad t = \beta - e \cdot c, \quad u_{\gamma} = \delta_{\gamma} + r \cdot m_{\gamma} \cdot c$
4. Formats the generated proof payload $(\pi, m_{\text{null}}, 0x\text{UserAddr})$ and submits it to the **Verifier** contract.

Current Status

- **Public Parameter Component:** $H_{\text{null}}, X, \vec{m}$
 - **Verifier Contract Context:** $\langle \mathcal{N}, 0x\text{UserAddr}, m_{\text{null}}, \pi \rangle$.
- **Signer Scope:** $x, \langle \text{UserID}, \mathcal{C}_2, \sigma' \rangle$
- **User Scope:** $0x\text{UserAddr}, m_{\text{null}}, m_{\gamma}, \lambda, \mathcal{N}, \sigma', \pi$

4.5.5 Phase 5: On-chain Verification and Minting (Verify)• **Verifier:**

1. Reconstructs the public commitment segment from the parameters $J = \{0, 1, \dots, \ell - 1, \text{null}, \text{Addr}\}$ revealed on-chain:

$$\mathcal{C}_J = G_1 + \sum_{i=0}^{\ell-1} m_i \cdot H_i + m_{\text{null}} \cdot H_{\text{null}} + m_{\text{Addr}} \cdot H_{\text{Addr}}$$

2. **Nullifier Security Assertions:**

- Computes the nullifier hash $\mathcal{N} = \text{Hash}(0x\text{UserAddr} \parallel m_{\text{null}})$.
- Validates its unspent status (`Used[N] == false`) to prevent double-spending.

3. **Calculates NIZK Confirmations:**

- (a) Recalculates the challenge scalar from the Fiat-Shamir context:

$$c' = H(\text{ctx} \parallel \vec{m}_J \parallel \bar{A} \parallel \bar{B} \parallel U)$$

- (b) **Pairing Equations** to check the signature binding relations:

$$e(\bar{A}, X) \stackrel{?}{=} e(\bar{B}, G_2)$$

- (c) **Homomorphic Structural Integrity** verification for algebraic consistency:

$$U + c' \cdot \bar{B} \stackrel{?}{=} s \cdot \mathcal{C}_J + t \cdot \bar{A} + u_\gamma \cdot H_\gamma$$

Accept the proof if and only if these equivalences hold true.

4. **Smart Contract Execution:**

- Updates the state registry by recording `Used[N] = true`.
- Triggers the creation function `mintSBT(0xUserAddr, attributes)` to assign the verifiable credential.

Current Status

- **Public Parameter Component:** $H_{\text{null}}, X, \vec{m}$
 - **Verifier Contract Context:** $\langle \mathcal{N}, 0x\text{UserAddr}, m_{\text{null}}, \pi, \text{TokenID} \rangle$.
- **Signer Scope:** $x, \langle \text{UserID}, \mathcal{C}_2, \sigma' \rangle$
- **User Scope:** $0x\text{UserAddr}, m_{\text{null}}, m_\gamma, \lambda, \mathcal{N}, \sigma', \pi, \text{TokenID}$

5 Security Analysis

In this section, we analyze the security properties of our proposed scheme and address potential vulnerabilities, ensuring unforgeability, unlinkability, and resistance against blockchain-specific attacks.

We evaluate our scheme within the standard provable security framework. Let $\lambda \in \mathbb{N}$ denote the security parameter, which concretely corresponds to the bit-length of the prime order p of the underlying cryptographic groups $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$ (i.e., $|p| \approx \lambda$). It fundamentally dictates the size of the key space and the mathematical hardness of the underlying algebraic problems (e.g., Discrete Logarithm, q -SDH).

A function $\text{negl}(\lambda)$ is termed *negligible* if, for every positive polynomial $\text{poly}(\cdot)$, there exists an integer N_c such that for all $\lambda > N_c$, it holds that $|\text{negl}(\lambda)| < \frac{1}{\text{poly}(\lambda)}$. Because

the best-known algorithms to solve the underlying hard problems on pairing-friendly curves (such as Pollard’s rho) require exponential time bounded by $\mathcal{O}(2^{\lambda/2})$, the success probability (Advantage, Adv) of any Probabilistic Polynomial-Time (PPT) adversary $\mathcal{A}$ evaluating within polynomial time is inherently asymptotically bounded by $\text{negl}(\lambda)$. A scheme is considered formally secure if $\text{Adv}(\mathcal{A}) \leq \text{negl}(\lambda)$.

5.1 Existential Unforgeability against Chosen Message Attacks (EUF-CMA)

Unforgeability ensures that no adversary, including unauthorized users or colluding external attackers, can compute a valid credential proof without the explicit authorization of the *Signer*.

Definition 1 (EUF-CMA Game): The challenger initializes the system parameters and provides the public key X to the adversary $\mathcal{A}$. $\mathcal{A}$ can query the signing oracle $\mathcal{O}_{\text{Sign}}$ for adaptively chosen vectors $\vec{m}_i$. Finally, $\mathcal{A}$ outputs a forgery: an NIZK proof π^* for a message vector $\vec{m}^*$. $\mathcal{A}$ wins if π^* passes the Verify algorithm and $\vec{m}^*$ was not previously queried to $\mathcal{O}_{\text{Sign}}$.

Theorem 1. *The BBS-DID scheme is EUF-CMA secure under the q -Strong Diffie-Hellman (q -SDH) assumption in $\mathbb{G}_1$ and $\mathbb{G}_2$. For any PPT adversary $\mathcal{A}$, its advantage is bounded by:*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{q\text{-SDH}}(\lambda) + \text{negl}(\lambda)$$

Proof Sketch. We construct a reduction algorithm $\mathcal{B}$ that solves the q -SDH problem using $\mathcal{A}$. If $\mathcal{A}$ produces a valid proof π^* for an unqueried message $\vec{m}^*$ with non-negligible probability, $\mathcal{B}$ can use the NIZK Extractor (detailed in Section 5.3) to extract the underlying forged signature (A^*, e^*) and the hidden variables $(m_{\text{null}}^*, m_{\gamma}^*, m_{\text{Addr}}^*)$. Due to the strong unforgeability of the core BBS signatures [TZ23], computing this valid tuple (A^*, e^*) allows $\mathcal{B}$ to break the q -SDH assumption. Since the q -SDH assumption holds in pairing-friendly curves, $\text{Adv}_{\mathcal{B}}^{q\text{-SDH}}(\lambda)$ is negligible, proving our construction’s unforgeability.

5.2 Signer-Blindness and On-Chain Unlinkability

Unlinkability ensures that even a fully compromised Signer, possessing a comprehensive transcript of all off-chain issuing requests, cannot link an on-chain NIZK proof back to a specific user.

Definition 2 (Unlinkability Game): The adversary $\mathcal{A}$ (acting as a malicious Signer) interacts with two honest users U_0 and U_1 , who request credentials using commitments $\mathcal{C}_{2,0}$ and $\mathcal{C}_{2,1}$. Both users successfully obtain partial signatures. The challenger randomly selects a bit $b \xleftarrow{\$} \{0, 1\}$ and generates an on-chain proof π_b utilizing the credentials of U_b . $\mathcal{A}$ is given π_b and outputs a guess b' . $\mathcal{A}$ wins if $b' = b$.

Theorem 2. *The proposed partially blind signature mechanism achieves computational signer-blindness and unlinkability limit. The advantage of $\mathcal{A}$ is defined as:*

$$\text{Adv}_{\mathcal{A}}^{\text{Blind}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

Proof Sketch. In the Apply phase, the user masks the commitment array $\mathcal{N}'$ with a uniformly distributed random blinding factor $\lambda \xleftarrow{\$} \mathbb{Z}_p$, yielding $\mathcal{C}_2 = \lambda \cdot \mathcal{N}'$. Because λ is chosen uniformly at random, $\mathcal{C}_2$ operates as an information-theoretically hiding commitment to the Signer. Furthermore, during the Verify phase, the constructed NIZK proof π is perfectly zero-knowledge. The randomization scalar r applied to $\bar{A} = r \cdot A$ statically scrambles the underlying algebraic structures. Thus, the probability distribution of the elements in π_b is statistically independent of the identity-binding parameters, ensuring $\mathcal{A}$ merely evaluates random variables, reducing its guessing probability exactly to $1/2$.

5.3 Computational Soundness of NIZK Protocols

This property guarantees that a malicious user cannot fabricate a valid proof π without knowing the actual underlying witnesses $(A, e, m_{\text{null}}, m_{\gamma}, m_{\text{Addr}})$.

Theorem 3. *The NIZK scheme employed in BBS-DID is computationally sound in the Random Oracle Model (ROM) under the General Discrete Logarithm (DL) assumption.*

Proof Sketch. By applying the General Forking Lemma, if an adversary $\mathcal{A}$ can generate a valid proof π for a false statement with a non-negligible advantage $\epsilon > 0$, we can construct a Knowledge Extractor $\mathcal{E}$. $\mathcal{E}$ runs $\mathcal{A}$ multiple times, rewinding the random oracle $H(\text{ctx} \parallel \dots)$ to reply with different challenge values c and c' . By obtaining two valid transcripts for the same initial commitments $(U, \bar{A}, \bar{B})$, the extractor elegantly solves the corresponding linear equations to deduce the discrete logarithms. Since the Generalized DL problem is assumed hard in $\mathbb{G}_1$, such adversary $\mathcal{A}$ cannot exist. Hence, $\text{Adv}_{\mathcal{A}}^{\text{Soundness}}(\lambda) \leq \text{negl}(\lambda)$.

5.4 Non-Transferability and Sybil Resistance

Blockchain applications mandate strict defense mechanisms against token commodification and Sybil identities.

Theorem 4. *The BBS-DID formulation guarantees non-transferability and double-spending resistance, provided the cryptographic hash function acts as a Collision-Resistant Pseudorandom Function (CR-PRF).*

Proof Sketch. To bypass non-transferability, an adversary must generate a valid NIZK proof mapped to an unverified address $m'_{\text{Addr}} \neq m_{\text{Addr}}$. Doing so while preserving the verification equation:

$$\begin{aligned} U + c' \cdot \bar{B} &= s \cdot \mathcal{C}_J + t \cdot \bar{A} \\ &+ u_{\gamma} \cdot H_{\gamma} \end{aligned} \tag{1}$$

requires finding an algebraic collision within the commitment matrices, which is reducible to the Discrete Logarithm problem, yielding $\text{Adv}_{\mathcal{A}}^{\text{Transfer}}(\lambda) \leq \text{negl}(\lambda)$.

Simultaneously, regarding double-spending and Sybil resistance, the contract unconditionally logs the generated nullifier $\mathcal{N} = \text{Hash}(0x\text{UserAddr} \parallel m_{\text{null}})$. Because the hash functions fundamentally act as a one-way permutation, finding two distinct sets of inputs that map to the identical $\mathcal{N}$ directly violates the collision resistance property, constraining $\text{Adv}_{\mathcal{A}}^{\text{Sybil}}(\lambda) \leq \text{negl}(\lambda)$. Thus, each valid credential strictly equates to a single decentralized identity token, mitigating any systemic multiplicity attacks.

6 Performance Evaluation

In this section, we evaluate the performance of the proposed scheme, focusing on off-chain computational overhead and on-chain verification costs, using a Rust-based WebAssembly (WASM) implementation and Solidity smart contracts.

6.1 Experimental Setup

Benchmarks were conducted on a consumer-grade machine (Apple M5 CPU, 16GB RAM, macOS Tahoe v26.4.1).

The off-chain cryptographic primitives are implemented in Rust (v1.95.0) and compiled into WebAssembly (WASM) via `wasm-bindgen` (v0.14.0) for execution within browser environments. The on-chain verification smart contracts are authored in Solidity (v0.8.33) and profiled using the Foundry framework, with the compiler optimizer enabled (200 runs).

Our implementation operates over the BN254 pairing-friendly elliptic curve to ensure EVM compatibility. This allows on-chain verification to directly leverage Ethereum's

native precompiled contracts (0x06, 0x07, and 0x08) for elliptic curve point addition, scalar multiplication, and optimal Ate pairing checks. The complete source code is open-sourced and publicly available at <https://github.com/letsverify>.

6.2 Off-Chain Computational Overhead

We benchmark the latency of the off-chain cryptographic primitives in two environments: a native Rust runtime (representing backend entities like Issuers and Verifier nodes) and a WebAssembly (WASM) engine (representing client applications like web browsers). Benchmarks are conducted over varying numbers of attributes, specifically $|M| \in \{5, 10, 20\}$.

Figure 3 presents the timing metrics for the key cryptographic phases: key pair generation, interactive signature issuance (user blind commitment setup and issuer partial signing), non-interactive zero-knowledge proof (PoK) generation, and proof verification.

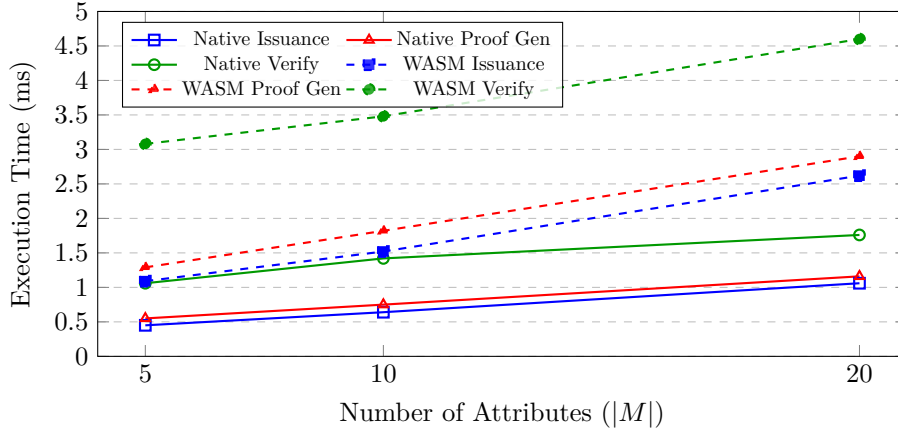


Figure 3: Off-Chain Execution Time of Cryptographic Phases across Attributes

The empirical results in Figure 3 indicate that algorithm performance scales linearly with the number of attributes $|M|$, except for the Key Generation phase which requires constant $O(1)$ time. The WASM execution incurs a $2.5\times$ to $3\times$ performance degradation compared to the native implementation due to browser sandbox overhead. Nonetheless, the maximum observed latency for proofs with 20 attributes remains under 5 milliseconds. This low processing overhead demonstrates the practical feasibility of the scheme for decentralized identity applications.

6.3 On-Chain Verification Cost

We analyze the gas consumption of the Solidity smart contracts deployed on the Sepolia testnet. The on-chain verification relies on the EVM BN254 precompiled contracts to evaluate the zero-knowledge proofs.

6.3.1 Contract Deployment Costs

The system logic is decoupled into two primary contracts: 1. **BBSVerifier**: A stateless contract containing the cryptographic pairing logic required to validate the NIZK proofs. 2. **IssuerVerifier**: The business logic contract responsible for enforcing credential rules, verifying nullifier uniqueness, and minting the resulting Soulbound Tokens (ERC-721/ERC-5484).

Deploying the **BBSVerifier** consumes approximately 1.63 million gas, while the **IssuerVerifier** requires approximately 3.20 million gas. The total deployment overhead is roughly 4.83 million gas. Because deployment is a one-time procedure typically executed by credential issuers, this cost is a reasonable fixed expense.

6.3.2 Proof Verification Gas Cost

On-chain proof verification algorithm requires validating a bilinear pairing equation and checking homomorphic elliptic curve consistency. Table 1 details the gas consumption required to verify proofs with varying numbers of attributes, detailing the exact count of EVM precompiled contract invocations.

Table 1: On-Chain Proof Verification Gas Cost and Operation Breakdown

Attributes ($ M $)	Total Gas	Est. Cost (USD)*	ecPairing	ecMul	ecAdd
5	256,007	\$0.0619	1	9	8
10	319,157	\$0.0796	1	14	13
20	445,948	\$0.1076	1	24	23

*Estimated fiat costs calculated based on prevailing Ethereum network rates as of May 6, 2026, 19:00 UTC+8.

We observe that the gas cost comprises a fixed overhead and a linearly scaling variable cost. The fixed overhead is dominated by the optimal Ate pairing check (**ecPairing** at 0x08), consuming approximately 112,000 gas for two pairings, along with state hashing and memory allocations. The variable cost scales with the number of attributes ($|M|$): aggregating the commitment sequence requires one elliptic curve scalar multiplication (**ecMul** at 0x07, $\sim 6,000$ gas) and one point addition (**ecAdd** at 0x06, ~ 150 gas) per attribute. Analytically, verifying a proof of $|M|$ attributes requires exactly 1 **ecPairing**, $|M| + 4$ **ecMul**, and $|M| + 3$ **ecAdd** operations.

Verifying a credential with 20 attributes incurs a total cost of $\sim 446k$ gas (approximately \$0.108 USD). This indicates that the proof verification remains well within Ethereum block limits and offers reasonable transaction fees for decentralized identity applications.

7 Conclusion

In this paper, we proposed BBS-DID, a verifiable credential framework designed to support privacy-preserving identity management within the Web3 ecosystem. By combining partially blind BBS signatures with Non-Interactive Zero-Knowledge (NIZK) proofs, our scheme addresses the practical conflict between public on-chain verification and user privacy. The introduction of a deterministic nullifier tied to Soulbound Tokens (SBTs) prevents Sybil attacks and guarantees non-transferability without compromising on-chain unlinkability. Our evaluations on EVM-compatible networks confirm the system’s practicality: leveraging native cryptographic precompiles keeps on-chain gas costs low, even when verifying credentials with up to 20 attributes.

While full anonymity protects users, it also presents challenges regarding accountability. For future work, we plan to investigate conditional traceability to address the potential for malicious behavior. A promising approach involves integrating Time-Lock Puzzles (TLP) or delay encryption [RSW96, MT19], where a user obscures a unique identity secret and submits it to the issuer during credential issuance. Under normal conditions, the user remains anonymous. However, upon detecting severe protocol violations, a designated entity (e.g., the issuer or a DAO) can expend predefined computational resources to solve the puzzle. This unlocks the secret, enabling the tracing and revocation of the offending user. Such a mechanism provides a configurable layer of accountability, balancing necessary privacy features with overall system integrity.

References

- [BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matt Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, pages 41–55, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
- [BSCG⁺14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE symposium on security and privacy*, pages 459–474. IEEE, 2014.
- [CDL16] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous attestation using the strong diffie hellman assumption revisited. Cryptology ePrint Archive, Paper 2016/663, 2016. URL: <https://eprint.iacr.org/2016/663>.
- [CL04] Jan Camenisch and Anna Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In Matt Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, pages 56–72, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
- [DKL⁺23] Jack Doerner, Yashvanth Kondi, Eysa Lee, Abhi Shelat, and LaKyah Tyner. Threshold bbs+ signatures for distributed anonymous credential issuance. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 773–789, 2023. doi:10.1109/SP46215.2023.10179470.
- [ENR⁺26] Liam Eagen, Hy Ngo, Vikas Rushi, Ying Tong, Moven Tsai, and Janabel Xia. OpenAC: Open design for transparent and lightweight anonymous credentials. Cryptology ePrint Archive, Paper 2026/251, 2026. URL: <https://eprint.iacr.org/2026/251>.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO’ 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [LCKO19] Jiwon Lee, Jaekyoung Choi, Jihye Kim, and Hyunok Oh. SAVER: SNARK-friendly, additively-homomorphic, and verifiable encryption and decryption with rerandomization. Cryptology ePrint Archive, Paper 2019/1270, 2019. URL: <https://eprint.iacr.org/2019/1270>.
- [MT19] Giulio Malavolta and Sri Aravinda Krishnan Thyagarajan. Homomorphic time-lock puzzles and applications. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 620–649, Cham, 2019. Springer International Publishing.
- [PSS19] Alexey Pertsev, Roman Semenov, and Roman Stromberg. Tornado cash: Privacy on ethereum. Tornado Cash Whitepaper. <https://tornadoeth.cash/r/whitepaper-tornado-cash>, 2019.
- [RS⁺22] Drummond Reed, Markus Sabadello, et al. Decentralized identifiers (DID) v1.0. W3C Recommendation, 2022. URL: <https://www.w3.org/TR/did-core/>.

-
- [RSW96] Ronald L. Rivest, Adi Shamir, and David A. Wagner. Time-lock puzzles and timed-release crypto. Technical Report Technical memo MIT/LCS/TR-684, MIT Laboratory for Computer Science, 1996. (Revision 3/10/96).
- [SLL⁺22] Manu Sporny, Dave Longley, Chad Lemire, et al. Verifiable credentials data model v1.1. W3C Recommendation, 2022. URL: <https://www.w3.org/TR/vc-data-model/>.
- [TZ23] Stefano Tessaro and Chenzhi Zhu. Revisiting BBS signatures. Cryptology ePrint Archive, Paper 2023/275, 2023. URL: <https://eprint.iacr.org/2023/275>.
- [WOB22] E Glen Weyl, Puja Ohlhaver, and Vitalik Buterin. Decentralized society: Finding web3’s soul. *Available at SSRN 4105763*, 2022.
- [ZLB24] Yong Zhou, Hong Lei, and Zijian Bao. Eisdspa: An efficient and secure blockchain-based donation scheme with privacy protection and auditability. *IEEE Open Journal of the Communications Society*, 5:7498–7510, 2024. doi: [10.1109/OJCOMS.2024.3504403](https://doi.org/10.1109/OJCOMS.2024.3504403).